Learning a function with a neural network

A first example: $\sin(x)$ **on** $[0, \pi]$

Directed Reading Program

Data Architecture Loss Training Testing

A neural network is a family of functions

$$\boxed{\widehat{y} = f_\theta(x)}$$

- x is an input and $\widehat{y}$ is the predicted output.
- θ collects the adjustable weights and biases.
- The architecture tells us how to combine these parameters.
- Learning means choosing θ so predictions fit examples.

Today: one real input, one real output, and a known target $y = \sin(x)$.

Inside one hidden neuron

$$z = wx + b$$

$$h = \tanh(z)$$

For example, if $w = 2$, $b = -1$, and $x = 0.5$, then $z = 0$ and $h = 0$.

- The weight w scales the input.
- The bias b shifts the response.
- The activation tanh introduces a nonlinear shape.

Training changes w and b. The activation formula stays fixed.

Our network has 49 trainable parameters

$$h_j(x) = \tanh(w_j x + b_j), \qquad j = 1, \ldots, 16$$

$$\boxed{f_\theta(x) = c + \sum_{j=1}^{16} v_j h_j(x)}$$

Part	Parameters	Meaning
Input to hidden layer	$16 + 16$	Weights w_j and biases b_j
Hidden to output layer	$16 + 1$	Weights v_j and bias c
Total	49	Numbers adjusted during training

The output is a weighted sum of 16 learned nonlinear responses.

Why the activation matters

If we remove tanh, the network becomes

$$f_\theta(x) = c + \sum_{j=1}^{16} v_j(w_j x + b_j)$$

$$= \underbrace{\left(\sum_j v_j w_j\right)}_{A} x + \underbrace{\left(c + \sum_j v_j b_j\right)}_{B}.$$

This is still a straight line: $Ax + B$.

Nonlinear activations let the network represent curved functions.

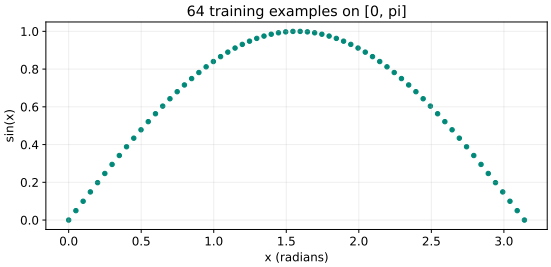
1. Training data

```
import torch
from torch import nn

torch.manual_seed(7)
x_train = torch.linspace(
    0, torch.pi, 64
).reshape(-1, 1)
y_train = torch.sin(x_train)
```

64 equally spaced inputs, including 0 and π. All angles use radians.

Shape `(64, 1)`: 64 examples with one input feature each.



2. Architecture in PyTorch

```
model = nn.Sequential(
    nn.Linear(1, 16),
    nn.Tanh(),
    nn.Linear(16, 1)
)
```

Operation	Batch shape
Input	64×1
First linear layer	64×16
Activation	64×16
Output layer	64×1

PyTorch initializes the weights and biases. They do not yet fit the data.

`Tanh` has no trainable parameters. The final layer has no activation.

3. Loss: mean squared error

$$L(\theta) = \frac{1}{64} \sum_{i=1}^{64} \Big(f_\theta(x_i) - y_i \Big)^2$$

```
loss_fn = nn.MSELoss()
```

- Subtract the target from each prediction.
- Square each difference, then take the average.
- A smaller loss means closer agreement on these examples.

Zero training loss would mean matching all 64 training targets exactly.

PyTorch: Optimizing Model Parameters

Backpropagation is the chain rule

For one example, let $e = f_\theta(x) - y$, $\ell = e^2$, and $h_j = \tanh(w_j x + b_j)$.

$$\frac{\partial \ell}{\partial v_j} = 2e\, h_j$$

$$\frac{\partial \ell}{\partial w_j} = \underbrace{2e}_{\text{output error}} \underbrace{v_j}_{\text{output weight}} \underbrace{(1 - h_j^2)}_{\tanh\ \text{derivative}} \underbrace{x}_{\text{input}}$$

The derivative tells us how a small parameter change affects the loss.

For MSE, average these derivatives over the training examples.
`loss.backward()` calculates them automatically for all parameters.

PyTorch: Automatic Differentiation

Gradient descent adjusts the parameters

$$\boxed{\theta_{\text{new}} = \theta_{\text{old}} - \eta \nabla_\theta L}$$

```
optimizer = torch.optim.SGD(
    model.parameters(), lr=0.05
)
```

- A positive derivative makes that parameter decrease.
- The learning rate $\eta = 0.05$ controls the size of the update.
- We use every example at each step: full-batch gradient descent.

A large step can increase the loss. A small step can make learning slow.

PyTorch: Optimizing Model Parameters

4. The complete training loop

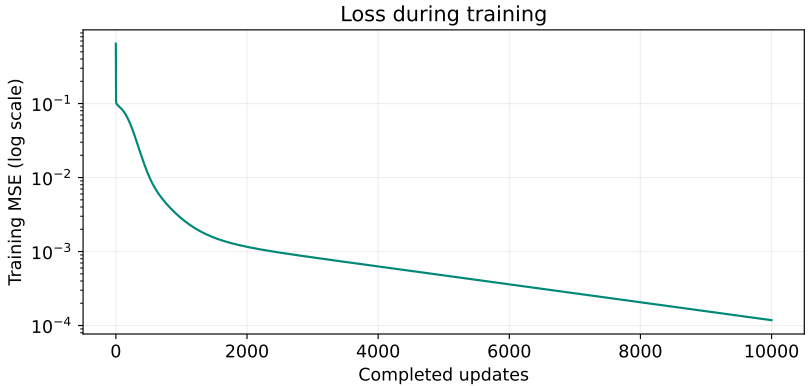
```
history = []
for epoch in range(10000):
    optimizer.zero_grad()
    loss = loss_fn(model(x_train), y_train)
    loss.backward()
    optimizer.step()
    history.append(loss.item())
```

`zero_grad()`	Clear gradients from the previous step.
`backward()`	Calculate gradients using the current parameters.
`step()`	Update the weights and biases.

One epoch uses all 64 examples. Here, each epoch has one update.

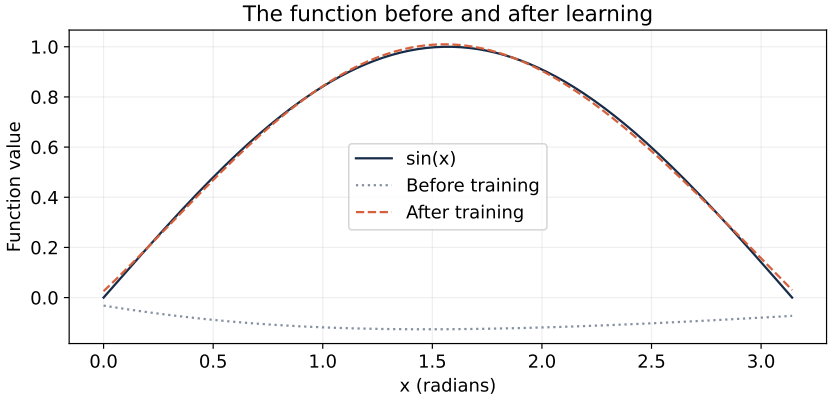
PyTorch: Optimizing Model Parameters

The training loss decreases



The vertical axis uses a logarithmic scale. Each stored loss precedes its update.
Final training MSE after 10,000 updates: 0.00011814.

The learned curve approximates sine



Only the 49 parameters changed. The model evaluates sums of tanh terms.
We use sine to generate targets and compare results.

5. Testing on new inputs

```
x_test = torch.tensor(
    [0.1, 0.3, 0.6, 0.9, 1.2, 1.5]
).reshape(-1, 1)
y_test = torch.sin(x_test)
model.eval()
with torch.no_grad():
    y_pred = model(x_test)
error = (y_pred - y_test).abs()
test_mse = loss_fn(y_pred, y_test).item()
```

All six inputs lie in $[0, \pi/2]$ and differ from the training inputs.

`no_grad()` skips gradient tracking. No optimizer update occurs.

Predictions compared with exact values

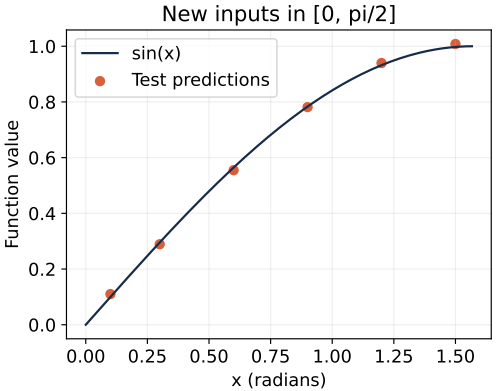
x (radians)	$\sin(x)$	Prediction	Absolute error
0.10	0.09983	0.11003	0.01019
0.30	0.29552	0.28904	0.00648
0.60	0.56464	0.55484	0.00980
0.90	0.78333	0.78145	0.00188
1.20	0.93204	0.93977	0.00774
1.50	0.99749	1.00802	0.01053

Test MSE: 0.00006936

Maximum absolute error at these six points: 0.01053.

Values come from the executed notebook. Table entries round to five decimal places.

Small errors are easier to see separately





Left: predictions against sine. Right: absolute error at the same six inputs.
Visual overlap alone does not mean the predictions are exact.

What this experiment establishes

- Training fits the sampled values by adjusting weights and biases.
- The six new points check interpolation inside the training interval.
- Six points cannot establish accuracy everywhere in the interval.
- The experiment does not establish accuracy beyond $[0, \pi]$ or teach periodicity explicitly.

If test results guide your choice of settings, reserve fresh test data for the final evaluation.

Experiments for discussion

1. Remove `nn.Tanh()`. What shape can the model learn?
2. Use 100 or 1,000 epochs. How does the fit change?
3. Use 4 hidden neurons instead of 16. Change both linear layers.
4. Plot on $[0, 2\pi]$. Does the model reproduce the next half-wave?

Restart the kernel and run all cells for each new experiment.

Companion notebook: `learning_sine.ipynb`

Further reading: PyTorch optimization and automatic differentiation tutorials.